



A TERMÉSZET TISZTA EREJE *a bőröd szépségéért*

KECSKETEJ – A TERMÉSZET GONDOSKODÁSA



TERMÉSZETES
és kíméletes



ÉRTÉKES
tápanyagokban
gazdag



KÍMÉLETES
ápolás minden
bőrtípusra



TISZTA
felesleges adalékok
nélkül



Kecsquetejes és Kenderolajos Termékek Webshop

Szoftverfejlesztő és -tesztelő projektfeladat

Juhász-Kókai Henriett
Szarka Andrea
Mitrikné Kovács Zsuzsa

2026.09.10.

Tartalomjegyzék

1. Projektleírás	4
1.1. Projekt célja.....	4
1.2. Projekt rövid leírása.....	4
1.3. Csapatmunka és feladatmegosztás.....	4
2. Rendszerterv	5
2.1. Funkcionális követelmények.....	5
2.2. Nem funkcionális követelmények.....	5
2.3. Szerepkörök és jogosultságok.....	5
2.4. Rendszerarchitektúra	5
2.4.1. Frontend réteg	5
2.4.2. Backend réteg.....	6
2.4.3. Adatbázis réteg	6
2.4.4. Hitelesítés és jogosultságkezelés	6
2.4.5. Kommunikáció a rétegek között	6
2.4.6. Email küldés.....	6
2.4.7. Rendszerarchitektúra.....	7
2.5. Adatmodell	8
3. Fejlesztési dokumentáció	10
3.1. Backend fejlesztés.....	10
3.1.1. Technológiai stack	10
3.1.2. Projektstruktúra.....	10
3.1.3. Adatbázis.....	11
3.1.4. Kapcsolódás (adatbázis)	11
3.1.5. API végpontok.....	11
3.1.5.1. ProductController – /Product	11
3.1.5.2. AuthController – /Auth	12
3.1.5.3. OrderController – /Order.....	12
3.1.6. Hitelesítés és jogosultságkezelés.....	12
3.1.7. CORS (Cross-Origin Resource Sharing)	13
3.1.8. Middleware pipeline (Program.cs)	13
3.1.9. Email értesítési szolgáltatás	14
3.1.10. Hibakezelés.....	14
3.1.11. Ismert korlátok és továbbfejlesztési lehetőségek.....	14
3.2. Frontend fejlesztés	15
3.2.1. Áttekintés.....	15
3.2.2. Technológiai stack	15

3.2.3. Projektstruktúra.....	15
3.2.4. Oldalak bemutatása	16
3.2.5. Backend-kapcsolódás.....	16
3.2.6. Kosár-kezelés	17
3.2.7. Hitelesítés-kezelés a frontenden.....	17
3.2.8. Admin felület — termékek kezelése (CRUD)	18
3.2.9. Egyéb funkciók.....	18
3.2.10. Ismert korlátok és továbbfejlesztési lehetőségek.....	19
3.2.11. Hibakezelés a frontenden	19
3.3. Adatbázis.....	20
3.3.1. Táblák szerkezete	20
3.3.2. Kapcsolatok (FOREIGN KEY megkötések)	20
3.3.4. ER-diagram.....	21
3.3.5. Adatműveletek (CRUD) bemutatása.....	21
3.3.6. Tranzakciókezelés	22
3.3.7. Biztonsági és adatintegritási szempontok	22
3.3.8. Továbbfejlesztési lehetőségek:	22
3.4 Verziókezelés	22
4. Tesztelés.....	23
4.1. Tesztesetek	23
4.3. Hibák listája.....	26
5. Felhasználói kézikönyv.....	27
5.1 Bevezetés.....	27
5.2 Kezdőlap	27
5.3 Termékek böngészése	28
5.4. Rólunk.....	29
5.5. Elérhetőségek.....	29
5.6. Általános feltételek.....	30
5.7. Regisztráció és bejelentkezés.....	30
5.8. Kosár használata	31
5.9. Rendelés leadása	32
5.10. Admin funkciók	33

1. Projektleírás

1.1. Projekt célja

Projektfeladatunk keretében egy valós vállalkozás számára terveztünk és fejlesztettünk webshopot. A vállalkozás 2008 óta foglalkozik kecsketejes krémek forgalmazásával, kínálatát pedig 3 évvel ezelőtt kenderolaj-alapú kozmetikai termékekkel (krémek, balzsamok) bővítette. A két termékcsaládhoz eddig nem tartozott önálló, modern online értékesítési felület.

A projekt célja egy olyan webshop létrehozása volt, ahol a vásárlók böngészhetik és megrendelhetik ezeket a termékeket.

Célfelhasználók:

1. magánvásárlók, akik természetes bőrápolási termékeket keresnek
2. az üzlet adminisztrátora, aki a termékkínálatot kezeli

1.2. Projekt rövid leírása

A Weblap az alábbi főbb funkciókkal rendelkezik:

- felhasználói regisztráció és bejelentkezés (cookie-alapú munkamenettel)
- termékek böngészése kategóriák szerint, részletes termékadatokkal
- kosár funkció (böngészőben tárolt, mennyiség módosítható, tételek törölhetők)
- rendelés leadása, automatikus email-visszaigazolással a vásárlónak és értesítéssel az eladónak
- admin felület: termékek hozzáadása, módosítása, törlése

1.3. Csapatmunka és feladatmegosztás

A projekt egy 3 fős csapat – Henriett, Andrea és Zsuzsa – közös munkájának eredménye, amely a fejlesztés teljes folyamata során rendszeres közös megbeszélésekkel és páros programozással valósult meg.

A kezdeti szakaszban Henriett készítette el az első HTML oldalakat, amelyek a vázát jelentették a frontendnek, és biztosította a webshop tartalmi elemeit a képekkel, leírásokkal együtt. Eközben Zsuzsa és Andrea megtervezte és létrehozta az adatbázist, valamint kialakította a kapcsolatot a backend, a frontend és az adatbázis között, lefektetve ezzel a projekt technikai alapjait.

A továbbiakban a csapat közösen, rendszeres meetingeken és pair programminggel tervezte meg a felhasználói felületet, és hozta létre a hozzá tartozó JavaScript fájlokat, így mindenki átlátta a teljes rendszer működését, és a fontosabb tervezési döntések közösen születtek.

A fejlesztés előrehaladtával a hatékonyabb munkavégzés érdekében a csapattagok fókusza differenciálódott: Henriett elsősorban a backend fejlesztésére koncentrált, Zsuzsa és Andrea pedig a frontend továbbfejlesztését vitte tovább.

A vizsgadokumentáció elkészítésében mindhárman részt vettünk: a dokumentáció vázát Zsuzsa állította össze, majd mindenki megírta a saját szakterületéhez tartozó technikai leírást – Henriett a backendet, Andrea a frontendet, Zsuzsa pedig az adatbázist, teszteseteket és a felhasználói kézikönyvet. A dokumentáció végső összeállítását és egységesítését Zsuzsa végezte.

2. Rendszerterv

2.1. Funkcionális követelmények

1. A felhasználó regisztrálhat e-mail cím és jelszó megadásával
2. A felhasználó bejelentkezhet, és a munkamenete cookie-val marad fenn
3. A felhasználó megtekintheti a termékek listáját, és rákattintva a részletes adatokat
4. A felhasználó termékeket tehet a kosarába, módosíthatja a mennyiséget, törölhet tételeket
5. A felhasználó leadhatja a rendelést, ha be van jelentkezve
6. A rendelés leadásakor a rendszer emailt küld a vásárlónak (visszaigazolás) és az adminnak (értesítés)
7. A rendszer nem engedélyezhet olyan rendelést, amelyben egy termékből több darabot rendelnek meg, mint amennyi az adott pillanatban készleten van
8. Az adminisztrátor bejelentkezés után elérheti a termékkezelő felületet
9. Az adminisztrátor új terméket vihet fel, meglévőt szerkeszthet vagy törölhet

2.2. Nem funkcionális követelmények

1. A felület reszponzív, asztali és mobil eszközön is használható
2. A jelszavak titkosítva (hash-elve) kerülnek tárolásra az adatbázisban (BCrypt algoritmussal)
3. A bejelentkezés cookie-alapú session-nel történik, admin és sima felhasználó jogosultság megkülönböztetésével
4. A backend és frontend külön futnak, közöttük CORS-szal engedélyezett kommunikáció zajlik

2.3. Szerepkörök és jogosultságok

Szerepkör	Jogosultságok
Vásárló (regisztrált felhasználó)	Regisztráció, bejelentkezés, termékek böngészése, kosár kezelése, rendelés leadása
Adminisztrátor	A fentiek mellett: termékek hozzáadása, módosítása, törlése

2.4. Rendszerarchitektúra

A rendszer **háromrétegű (kliens-szerver) architektúrát** követ, ahol a frontend, a backend és az adatbázis egymástól elkülönülő, önálló komponensek, amelyek jól definiált interfészeken (REST API, SQL kapcsolat) keresztül kommunikálnak egymással.

2.4.1. Frontend réteg

- **Technológia:** HTML5, CSS3, natív JavaScript (ES6+), Bootstrap 4 (reszponzív rácsrendszer és UI komponensek)
- **Felépítés:** statikus, szerver oldali renderelés nélküli oldalak (Fooldal.html, Termékek.html, Kosar.html, Admin.html stb.), amelyek betöltéskor JavaScripttel (fetch API-val) kéri le a dinamikus adatokat a backendtől
- **Állapotkezelés:** a kosár tartalma kliens oldalon, a böngésző localStorage-ában tárolódik, így bejelentkezés nélkül is megmarad a kosár tartalma
- **Fejlesztői környezet:** VS Code + Live Server kiterjesztés, ami helyi kiszolgálón (127.0.0.1:5501) futtatja a statikus fájlokat

2.4.2. Backend réteg

- **Technológia:** ASP.NET Core Web API (.NET), C# nyelven, Kestrel webszerverrel futtatva (<https://localhost:7249>)
- **Rétegzett felépítés:**
 - **Controllers** — a HTTP kéréseket fogadó végpontok (AuthController, ProductController, OrderController, UserController)
 - **Models** — az adatbázis táblákat leképező osztályok (Users.cs, Products.cs)
 - **Dtos** (Data Transfer Objects) — a kérésekben/válaszokban utazó adatok szerkezetét definiáló osztályok (RegisterDto, LoginDto, ProductDto, OrderDto)
 - **Connect.cs** — az adatbázis-kapcsolat létrehozásáért felelős segédosztály
- **Adathozzáférés:** közvetlen ADO.NET-stílusú SQL lekérdezések a MySqlConnectionor NuGet csomagon keresztül (nem ORM/Entity Framework)
- **API stílus:** REST elveket követő, JSON alapú végpontok (pl. GET /Product, POST /Order, POST /Auth/login)

2.4.3. Adatbázis réteg

- **Technológia:** MySQL, helyi fejlesztés során WAMP szerveren futtatva, phpMyAdmin felülettel kezelve
- **Táblák:** Users, Products, Orders, OrderItems (részletesen a 2.5 Adatmodell fejezetben)

2.4.4. Hitelesítés és jogosultságkezelés

- **Módszer:** cookie-alapú munkamenet-kezelés (ASP.NET Core Cookie Authentication)
- **Jelszókezelés:** a jelszavak BCrypt algoritmussal hash-elve kerülnek tárolásra, soha nem sima szöveggént
- **Jogosultsági szintek:** a bejelentkezéskor a rendszer a felhasználó IsAdmin mezője alapján egy Admin szerepkör-claimet ad a session-höz, amit a védett végpontok ([Authorize(Roles = "Admin")]) ellenőriznek
- **Cookie beállítások:** SameSite=None és Secure=true, mivel a frontend és backend különböző origin-ről (portról) fut, így cross-origin kéréseknél is érvényes marad a bejelentkezés

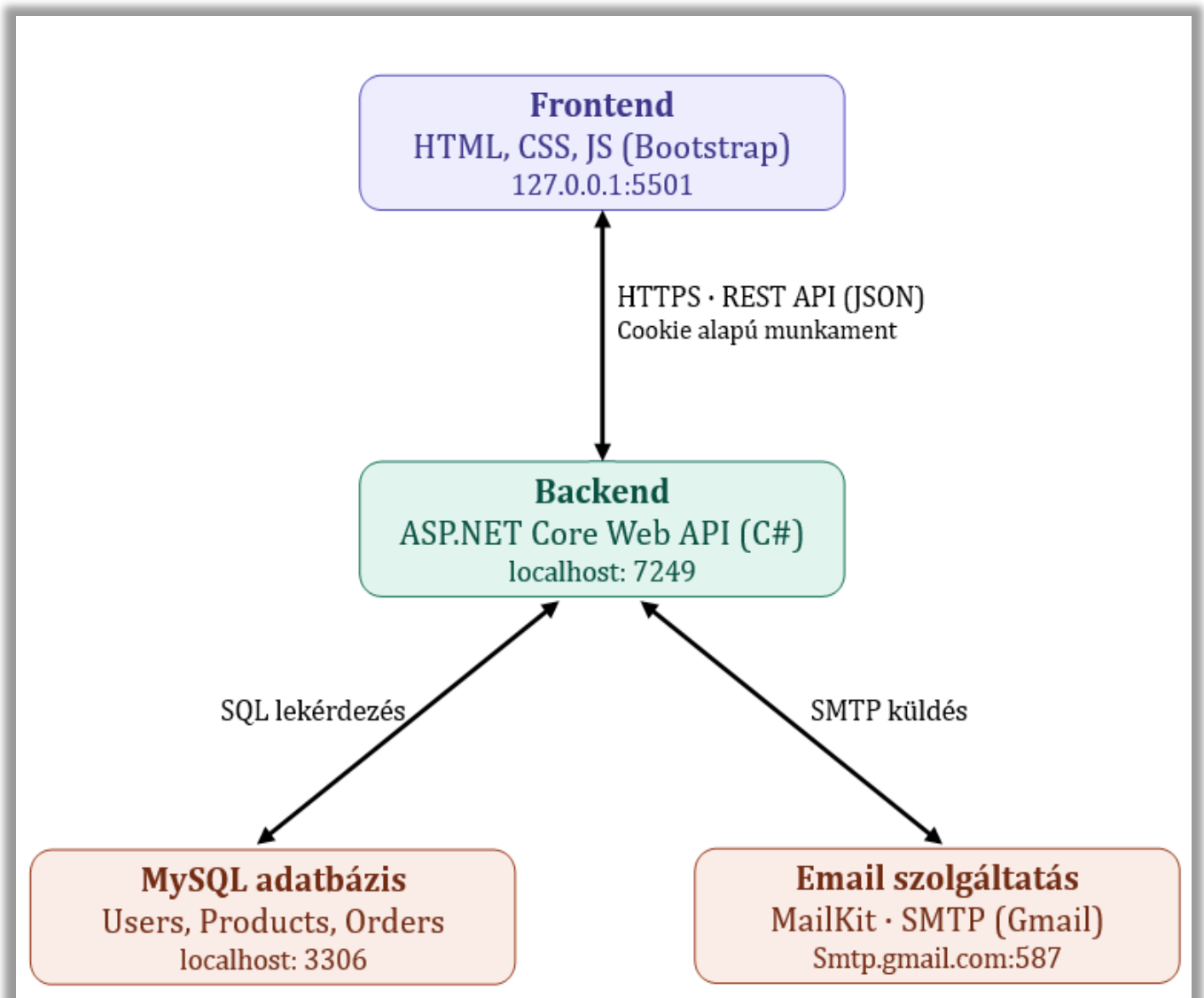
2.4.5. Kommunikáció a rétegek között

- **Frontend ↔ Backend:** HTTPS-en keresztüli REST hívások, JSON adatformátumban; mivel a két komponens különböző origin-en fut, a backend **CORS (Cross-Origin Resource Sharing)** szabályt alkalmaz, ami kifejezetten a frontend origin-jét (<http://127.0.0.1:5501>) engedélyezi, hitelesítő adatokkal (cookie) együtt
- **Backend ↔ Adatbázis:** közvetlen MySQL kapcsolat, minden kéréshez új, a kapcsolati stringből (Connect.cs) létrehozott adatbázis-kapcsolattal

2.4.6. Email küldés

- **Technológia:** MailKit könyvtár, SMTP protokollon keresztül, Gmail SMTP szerverrel (smtp.gmail.com:587, StartTls titkosítással)
- **Működés:** sikeres rendelés leadásakor a backend automatikusan két emailt küld: egy visszaigazolást a vásárlónak, és egy értesítést az adminisztrátor(ok)nak, a rendelés tételeivel és végösszegével

2.4.7. Rendszerarchitektúra



2.5. Adatmodell

Az adatbázis négy egymáshoz kapcsolódó táblából épül fel:

- a **Users tábla** tárolja a regisztrált felhasználók adatait (bejelentkezéshez és jogosultságkezeléshez)
- a **Products tábla** a webshopban elérhető termékeket
- az **Orders** és **OrderItems** táblák a leadott rendeléseket és azok tételeit rögzítik

A táblák közötti kapcsolatokat idegen kulcsok (foreign key) biztosítják: egy felhasználóhoz több rendelés, egy rendeléshez pedig több rendelési tétel tartozhat.

Users tábla		
Mezőnév	Típus	Leírás
id	INT	Azonosító (elsődleges kulcs)
username	VARCHAR(50)	Felhasználónév
email	VARCHAR(100)	Email cím (egyedi)
password	TEXT	Hash-elt jelszó (BCrypt)
createdat	TIMESTAMP	Regisztráció időpontja
updatedat	TIMESTAMP	Utolsó módosítás időpontja
isAdmin	TINYINT(1)	Admin jogosultság jelzése (0/1)

Products tábla		
Mezőnév	Típus	Leírás
id	INT	Azonosító (elsődleges kulcs)
category	VARCHAR(100)	Kategória
name	VARCHAR(150)	Termék neve
unit	VARCHAR(50)	Kiszerezés
price	INT	Ár (Ft)
description	TEXT	Termékleírás
image	VARCHAR(200)	Kép elérési útja
stock	INT	Készlet
created_at	TIMESTAMP	Létrehozás időpontja

Orders tábla		
Mezőnév	Típus	Leírás
id	INT	Azonosító (elsődleges kulcs)
user_id	INT	A rendelést leadó felhasználó azonosítója
total_price	INT	A rendelés végösszege (Ft)
status	VARCHAR(50)	Rendelés állapota (pl. "Új")
created_at	TIMESTAMP	Rendelés leadásának időpontja

OrderItems tábla		
Mezőnév	Típus	Leírás
id	INT	Azonosító (elsődleges kulcs)
order_id	INT	Rendelés azonosítója
product_id	INT	A termék azonosítója
product_name	VARCHAR(150)	A termék neve a rendelés időpontjában (rögzített másolat)
unit_price	INT	Egységár a rendelés időpontjában (Ft)
quantity	INT	Megrendelt mennyiség

Kapcsolatok a táblák között

A táblák InnoDB tárolómotort használnak, ami lehetővé teszi a valódi, adatbázis-szinten kikényszerített FOREIGN KEY megkötéseket:

Kapcsolat	Törléskori viselkedés	Indoklás
Orders.user_id → Users.id	ON DELETE CASCADE	Ha egy felhasználó törlődik, a rendelései is automatikusan törlődnek vele
OrderItems.order_id → Orders.id	ON DELETE CASCADE	Ha egy rendelés törlődik, a hozzá tartozó tételek is automatikusan törlődnek vele
OrderItems.product_id → Products.id	ON DELETE SET NULL	Ha egy terméket törölnek (admin funkció), a korábbi rendelések tétele megmarad — csak a hivatkozás válik üressé, mivel a product_name/unit_price mezők másolatként külön is tárolva vannak

Az OrderItems tábla tervezési döntése: a tábla a product_name és unit_price mezőket **rögzített másolatként** is tárolja, nem csupán a product_id-n keresztüli hivatkozásként. Ennek oka, hogy ha egy admin később módosítja egy termék nevét vagy árát a Products táblában, a **korábban leadott rendelések adatai ne változzanak meg visszamenőleg** — így a rendelési előzmények mindig azt az árat és nevet mutatják, ami a vásárlás pillanatában érvényes volt.

Az említett kódrészlet az OrderController.cs-ből:

```
var itemCmd = new MySqlCommand(
    "INSERT INTO OrderItems (order_id, product_id, product_name, unit_price, quantity) "
    + "VALUES (@orderId, @productId, @name, @price, @qty)",
    connection, (MySqlTransaction)transaction);
```

```
itemCmd.Parameters.AddWithValue("@name", item.Name);
itemCmd.Parameters.AddWithValue("@price", item.Price);
```

Biztonsági szempont: a Users tábla password mezője soha nem tartalmaz sima szöveges jelszót — a jelszavak BCrypt algoritmussal hash-elve kerülnek tárolásra, így egy esetleges adatbázis-hozzáférés esetén sem derülnek ki a felhasználók valódi jelszavai.

Egyediségi megkötés: a Users tábla email mezője egyedi (unique) kell legyen, hogy egy email címhez csak egy fiók tartozhasson, elkerülve a duplikált regisztrációkat.

3. Fejlesztési dokumentáció

3.1. Backend fejlesztés

A backend egy **ASP.NET Core Web API** alkalmazás, amely RESTful végpontokon keresztül szolgálja ki a frontendet (HTML/CSS/JavaScript), és **MySQL** adatbázisban tárolja az adatokat.

Fő funkciók:

- Termékek listázása, kezelése (CRUD)
- Felhasználói regisztráció és bejelentkezés
- Cookie-alapú hitelesítés és jogosultságkezelés
- Rendelés leadása bejelentkezett felhasználók számára
- Automatikus email értesítés rendelés leadásakor (vásárló + admin)

3.1.1. Technológiai stack

Réteg	Technológia
Backend keretrendszer	ASP.NET Core Web API (.NET 10)
Adatbázis	MySQL
Adatbázis-kapcsolat	MySqlConnection
Hitelesítés	Cookie-alapú autentikáció (CookieAuthenticationDefaults)
Jelszó-hash	BCrypt.Net
Email küldés	MailKit / MimeKit (SMTP – Gmail)
Fejlesztői környezet	Visual Studio
Frontend (kapcsolódó)	HTML, CSS (Bootstrap), natív JavaScript

3.1.2. Projektstruktúra

Webshop/

└─ Properties/	
└─ launchSettings.json	– indítási profilok (http / https / IIS Express)
└─ Controllers/	
└─ AuthController.cs	– regisztráció, bejelentkezés, munkamenet-ellenőrzés
└─ OrderController.cs	– rendelés leadása
└─ ProductController.cs	– termékek lekérdezése és kezelése
└─ Dtos/	
└─ LoginDto.cs	
└─ OrderDto.cs	
└─ ProductDto.cs	
└─ RegisterDto.cs	
└─ Models/	
└─ Products.cs	
└─ Users.cs	
└─ appsettings.json	– konfiguráció (adatbázis, email, stb.)
└─ Connect.cs	– adatbázis-kapcsolat kezelése
└─ EmailService.cs	– SMTP email küldés
└─ Program.cs	– alkalmazás konfiguráció, middleware pipeline
└─ Webshop.csproj	

3.1.3. Adatbázis

Az alkalmazás egy MySQL adatbázist használ (projectwebshop, localhost:3306), amihez a Connect.cs osztály biztosít kapcsolatot a MySqlConnection csomagon keresztül, SERVER, DATABASE, UID, PWD paraméterekkel.

Az adatbázis négy táblát tartalmaz: Users, Products, Orders, OrderItems. A táblák részletes szerkezetét, oszlopait és a köztük lévő kapcsolatokat lásd a 2.5. Adatmodell és a 3.3. Adatbázis fejezetekben.

Tervezési megjegyzés: az OrderItems tábla a product_name és unit_price mezőket rögzített másolatként is tárolja, nem csupán a product_id-n keresztüli hivatkozásként. Ez biztosítja, hogy egy későbbi terméktörlés vagy áremelés ne módosítsa visszamenőleg a már leadott rendelések adatait.

3.1.4. Kapcsolódás (adatbázis)

A Connect.cs osztály minden kéréshez egy új MySqlConnection példányt hoz létre a GetConnection() metóduson keresztül, amit a controllerek using blokkban használnak, biztosítva a kapcsolat automatikus lezárását.

Az említett kódrészlet:

```
public MySqlConnection GetConnection()
{
    var connection = new MySqlConnection(ConnectionString);
    connection.Open();
    return connection;
}
```

Ez a minta minden controllerben ismétlődik: minden végpont-hívás saját, önálló kapcsolatot nyit és zár, ami egyszerű és biztonságos megközelítés kisebb terhelésű rendszereknél.

3.1.5. API végpontok

3.1.5.1. ProductController – /Product

Metódus	Végpont	Jogosultság	Leírás
GET	/Product	Nyilvános	Összes termék listázása
GET	/Product/{id}	Nyilvános	Egy termék lekérdezése azonosító alapján
POST	/Product	Admin	Új termék létrehozása
PUT	/Product/{id}	Admin	Termék módosítása
DELETE	/Product/{id}	Admin	Termék törlése

A POST és PUT végpontok elutasítják a kérést, ha a megadott ár negatív.

3.1.5.2. AuthController – /Auth

Metódus	Végpont	Jogosultság	Leírás
POST	/Auth/register	Nyilvános	Regisztráció (felhasználónév, email, jelszó), jelszó BCrypt hash-eléssel
POST	/Auth/login	Nyilvános	Bejelentkezés (email + jelszó), sikeres hitelesítés esetén cookie kiállítása
POST	/Auth/logout	Bejelentkezett	Kijelentkezés, a munkamenet-cookie törlése
GET	/Auth/me	Bejelentkezett	Aktuális munkamenet ellenőrzése, felhasználói adatok visszaadása

A jelszavakat **BCrypt** algoritmussal hash-elve tárolja és ellenőrzi a rendszer (BCrypt.Net.BCrypt.Verify).

3.1.5.3. OrderController – /Order

Metódus	Végpont	Jogosultság	Leírás
POST	/Order	Bejelentkezett	Rendelés leadása a kosár tartalma alapján

Működés:

1. A hitelesített felhasználó azonosítóját a cookie-ban tárolt claim-ből olvassa ki (ClaimTypes.NameIdentifier).
2. Tranzakción belül létrehozza a rendelés fejkordját (Orders), majd a hozzá tartozó tételeket (OrderItems).
3. Sikeres mentés után lekérdezi a felhasználó email címét, és automatikus visszaigazoló emailt küld neki, valamint értesítést az adminisztrátori címre.
4. Hiba esetén a tranzakció visszagörgetésre kerül (RollbackAsync), így nem marad félkész rendelés az adatbázisban.
5. Rendelés rögzítése előtt a POST/Order végpont tranzakción belül ellenőrzi és zárolja az egyes termékek készletét. Elutasítja a kérést, ha a kért mennyiség meghaladja az elérhető, sikeres rendelés esetén pedig csökkenti a stock értékét.

3.1.6. Hitelesítés és jogosultságkezelés

A rendszer **cookie-alapú hitelesítést** használ (nem JWT token a kliens oldalon tárolva).

Az említett kódrészlet:

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.Cookie.Name = "WebshopAuth";
        options.Cookie.SameSite = SameSiteMode.None;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
        options.ExpireTimeSpan = TimeSpan.FromDays(7);
        options.SlidingExpiration = true;
        options.Events.OnRedirectToLogin = context =>
        {
            context.Response.StatusCode = 401;
            return Task.CompletedTask;
        };
    });
```

Kulcs jellemzők:

- A sikeres bejelentkezéskor a szerver egy WebshopAuth nevű, HttpOnly, Secure sütit állít ki, amit a JavaScript nem tud kiolvasni (XSS elleni védelem).
- A süti élettartama 7 nap, csúszó lejárat (minden aktív használat meghosszabbítja).
- Bejelentkezés nélküli hozzáférés esetén a szerver 401-es státuszkódot ad vissza (nem átirányítást egy login oldalra, mivel ez egy API).
- A frontend minden hitelesítést igénylő kérésnél a credentials: "include" beállítást használja, hogy a böngésző automatikusan elküldje a sütit.
- A védett végpontok [Authorize] attribútummal vannak megjelölve; adminisztrátori jogosultságot igénylő műveleteknél [Authorize(Roles = "Admin")] szerepel.

3.1.7. CORS (Cross-Origin Resource Sharing)

Mivel a frontend (Live Server, <http://127.0.0.1:5501>) és a backend (<https://localhost:7249>) különböző originről fut, explicit CORS-szabályzat szükséges:

Az említett kódrészlet:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("FrontendPolicy", policy =>
    {
        policy.WithOrigins("http://127.0.0.1:5501")
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials();
    });
});
```

Az `AllowCredentials()` elengedhetetlen ahhoz, hogy a hitelesítési süti átmenjen a kéréseken – ez csak akkor működik, ha az engedélyezett origin **konkrét cím** (nem * wildcard).

3.1.8. Middleware pipeline (Program.cs)

A kérés-feldolgozási lánc a következő sorrendben épül fel:

```
csharp
app.UseHttpsRedirection();
app.UseCors("FrontendPolicy");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Ez a sorrend kritikus:

1. **HTTPS átirányítás** – biztonságos kapcsolat kikényszerítése
2. **CORS** – engedélyezett originek ellenőrzése, mielőtt bármilyen hitelesítés történne
3. **Authentication** – a beérkező süti alapján azonosítja a felhasználót
4. **Authorization** – ellenőrzi, hogy a felhasználónak van-e jogosultsága az adott végponthoz
5. **Controllers** – a tényleges végpont-logika lefuttatása

3.1.9. Email értesítési szolgáltatás

Az EmailService.cs egy önálló, dependency injection-nel regisztrált szolgáltatás (AddScoped<EmailService>()), amely SMTP-n keresztül (Gmail) küld emaileket a **MailKit** könyvtár segítségével.

Konfiguráció (appsettings.json → EmailSettings szekció), kódrészlet:

```
"EmailSettings": {  
  "SmtpServer": "",  
  "SmtpPort": 0,  
  "SenderEmail": "...",  
  "SenderPassword": "...",  
  "AdminEmail": "...",  
  "MasodikEmail": "..."}  
}
```

Küldési folyamat:

1. Kapcsolódás a Gmail SMTP szerveréhez StartTls titkosítással.
2. Hitelesítés Gmail-alkalmazásjelszóval (a 2FA miatt szükséges, mivel a sima fiókjelszó nem használható SMTP-n).
3. Email összeállítása és elküldése (MimeMessage).

Küldött emailek típusai:

- **Vásárlói visszaigazolás** – a rendelés tételeivel és végösszegével, a rendelést leadó felhasználó email címére.
- **Admin értesítés** – új rendelés érkezéséről, a konfigurált admin email címre.

3.1.10. Hibakezelés

A controllerek konzisztensen a következő mintát követik:

- Sikeres művelet: 200 OK / 201 Created, JSON válasz üzenettel
- Nem található erőforrás: 404 Not Found
- Hiányzó/érvénytelen hitelesítés: 401 Unauthorized
- Hiba bemenet: 400 Bad Request
- Szerverhiba (pl. adatbázis-tranzakció sikertelensége): 500 Internal Server Error, a hiba üzenetével

A OrderController tranzakciókezelést alkalmaz (BeginTransactionAsync / CommitAsync / RollbackAsync), így egy rendelés mentése atomi művelet: vagy teljesen sikerül (fejrekord + minden tétel), vagy semmi nem kerül be az adatbázisba hiba esetén.

3.1.11. Ismert korlátok és továbbfejlesztési lehetőségek

- A kapcsolati adatok (Connect.cs) jelenleg fixen a kódban szerepelnek (localhost, root, üres jelszó) – érdemes lenne ezeket is az appsettings.json-ba, illetve éles környezetben környezeti változókba kiszervezni.
- Nincs vendég (bejelentkezés nélküli) rendelési lehetőség – ez tudatos tervezési döntés volt, csak regisztrált, bejelentkezett felhasználók adhatnak le rendelést.
- Új PaymentController egy fizetési szolgáltató API-jának integrálására
- OrderController bővítése rendelésállapot-kezeléssel
- Új végpontok a szállítási/számlázási adatok mentésére és lekérdezésére
- A Products tábla GetString() hívásai kivételt dobnának, ha bármely nem opcionális mező (category, name, unit, description) NULL értéket tartalmazna az adatbázisban – érdemes lenne NOT NULL megszorítást alkalmazni, vagy null-tűrő olvasást bevezetni.
- Rendelés-visszavonás, státuszkövetés (pl. „feldolgozás alatt”, „kiszállítva”) jelenleg nincs kiépítve a status mezőn túl.

3.2. Frontend fejlesztés

3.2.1. Áttekintés

A frontend statikus HTML oldalakból áll, amelyeket natív JavaScript és a Bootstrap keretrendszer egészít ki. Az oldalak betöltéskor, illetve felhasználói interakcióra fetch API hívásokkal kéri le és küldik az adatokat a backend REST API-jának (<https://localhost:7249>).

Fő funkciók:

- Termékek listázása és részletes megtekintése
- Kiemelt ("népszerű") termékek megjelenítése a kezdőlap
- Kosárkezelés bejelentkezés nélkül is (helyi tárolás)
- Felhasználói regisztráció, bejelentkezés, kijelentkezés
- Rendelés leadása bejelentkezett felhasználók számára
- Adminisztrátori felület a termékek kezeléséhez (CRUD)

3.2.2. Technológiai stack

Réteg	Technológia
Jelölőnyelv / stílus	HTML5, CSS3, Bootstrap 4 (reszponzív rácsrendszer és UI komponensek)
Klientsoldali logika	Natív JavaScript (ES6+), fetch API
Segédkönyvtárak	jQuery, Popper.js (a Bootstrap komponensek működéséhez)
Állapottárolás	Böngésző localStorage (kosár tartalma, bejelentkezési állapot cache-elése)
Backend-kommunikáció	REST API hívások credentials: "include" beállítással (cookie-alapú hitelesítéshez)
Fejlesztői környezet	Visual Studio Code + Live Server kiterjesztés (127.0.0.1:5501)

3.2.3. Projektstruktúra

FRONTEND/

└─ bootstrap/	– Bootstrap, jQuery és Popper könyvtárfájlok
└─ KEPEK/, Képek/	– termékekhez és egyéb tartalmakhoz tartozó képek
└─ kezdokepek/	– a főoldali/kezdőlap carousel képei
└─ szoveg/	– szöveges tartalmak
└─ Fooldal.html	– kezdőlap (bemutató, népszerű termékek)
└─ Termekek.html	– termékek listázása és részletes nézete
└─ Kosar.html	– kosár tartalma, rendelés leadása
└─ Admin.html	– bejelentkezés, regisztráció, admin felület
└─ Rolunk.html	– Rólunk oldal
└─ Elerhetosegek.html	– elérhetőségek
└─ Altalanosfeltetelek.html	– Általános Szerződési Feltételek
└─ index.css	– közös stíluslap
└─ indexBackend.js	– termékek lekérése és megjelenítése (Termekek.html)
└─ nepyszeruTermekek.js	– kiemelt termékek (Fooldal.html)
└─ kosar.js	– kosárkezelés (localStorage) és navigáció-állapot
└─ kosarOldal.js	– kosár táblázat megjelenítése, rendelés leadása
└─ admin.js	– belépés, regisztráció, admin termékkezelés (CRUD)

3.2.4. Oldalak bemutatása

Fooldal.html

A webshop kezdőlapja. Tartalmaz egy Bootstrap carouselt a kezdőképekkel, egy statikus bemutatkozó szöveget a vállalkozásról, valamint egy "Népszerű termékek" szekciót, amelyet a nepszeruTermekek.js tölt fel dinamikusan.

Termekek.html

Az összes termék listázására szolgáló oldal. Az indexBackend.js tölti be és jeleníti meg a termékkártyákat, valamint ez kezeli a termékek részletes nézetét (felugró/kibontott panel) is.

Kosar.html

A kosár tartalmát táblázatos formában megjelenítő oldal (kosarOldal.js), ahol a mennyiség módosítható, tételek törölhetők, és leadható a rendelés. Ha a felhasználó nincs bejelentkezve, az oldal helyben felkínálja a bejelentkezés lehetőségét, a kosár tartalmának elvesztése nélkül.

Admin.html

Egyetlen oldal, amely a bejelentkezési állapottól függően három nézet közül jelenít meg egyet: bejelentkezés/regisztráció űrlap, admin termékkezelő felület (ha a felhasználó adminisztrátor), vagy egy egyszerű üdvözlő nézet (sima, bejelentkezett felhasználó esetén). A logikát az admin.js valósítja meg.

Rolunk.html, Elerhetosegek.html, Altalanosfeltetelek.html

Statikus tartalmú, nem dinamikus adatokat megjelenítő oldalak.

3.2.5. Backend-kapcsolódás

A backend elérési útja minden érintett fájlban (indexBackend.js, nepszeruTermekek.js, kosar.js, admin.js) egy API_BASE nevű konstansban van rögzítve:

```
const API_BASE = "https://localhost:7249";
```

Minden kérés a fetch API-t használja, credentials: "include" beállítással, hogy a böngésző elküldje a hitelesítési cookie-t (WebshopAuth) a backend felé:

Az említett kódrészlet:

```
const response = await fetch(`${API_BASE}/Product`, {  
  method: "GET",  
  credentials: "include"  
});
```

3.2.6. Kosár-kezelés

A kosár tartalma nem a szerveren, hanem a böngésző localStorage-ában tárolódik (kosar.js), JSON formátumban. Ennek köszönhetően a kosár bejelentkezés nélkül is használható, és böngésző-újraindítás után is megmarad.

Fő funkciók:

Funkció	Leírás
getKosar() / mentKosar()	A kosár beolvasása / elmentése a localStorage-ba
kosarhozAd(termek)	Termék hozzáadása; ha már szerepel, a mennyiséget növeli
kosarbolTorol(id)	Tétel eltávolítása a kosárból
mennyisegModosit(id, ujMennyiseg)	Egy tétel darabszámának módosítása
kosarOsszesen()	A kosár teljes végösszegének kiszámítása
frissitKosarSzamlalo()	A navigációs sávban látható darabszám frissítése
mutasdToaster(uzenet)	Rövid, eltűnő értesítés megjelenítése (pl. "X bekerült a kosárba!")

3.2.7. Hitelesítés-kezelés a frontenden

A frontend a backend cookie-alapú hitelesítését használja ki; token nem tárol a kliens oldalon. A bejelentkezési állapotot a /Auth/me végpont lekérdezésével ellenőrzi.

A navigációs sáv dinamikus frissítése (kosar.js — frissitAuthNav)

A navigációs menü "Bejelentkezés" linkje automatikusan "Fiókom"-ra cserélődik, ha a felhasználó be van jelentkezve. A gyorsabb megjelenítés érdekében a funkció előbb egy localStorage-ban cache-elt állapotot jelenít meg, majd a backend tényleges válasza alapján szükség esetén frissíti azt.

Az Admin.html oldal három nézete (admin.js — checkAuthStatus)

- Nincs bejelentkezve → bejelentkezési / regisztrációs űrlap (egyetlen űrlap, amely a toggleFormMode() függvénnyel vált a két mód között)
- Bejelentkezett adminisztrátor → termékkezelő admin felület
- Bejelentkezett, nem admin felhasználó → egyszerű üdvözlő nézet

Bejelentkezés a kosár oldaláról

Ha a felhasználó bejelentkezés nélkül próbál rendelést leadni (a backend 401-es választ ad), a Kosar.html oldal helyben felkínálja a bejelentkezést, a kosár tartalmának és a folyamat megszakítása nélkül.

3.2.8. Admin felület — termékek kezelése (CRUD)

Az Admin.html admin nézete táblázatban listázza a termékeket (id, kategória, név, kiszerelés, ár, készlet), Szerkesztés és Törlés gombokkal soronként.

Funkció	HTTP metódus	Végpont	Leírás
Termékek betöltése	GET	/Product	Az összes termék lekérése és táblázatba rendezése
Termék mentése (új / szerk.)	POST / PUT	/Product, /Product/{id}	Ugyanaz a mentés-gomb kezeli mindkét esetet; a döntés a kitöltött id mezőn múlik
Termék törlése	DELETE	/Product/{id}	Megerősítő kérdés (confirm) után

Szerkesztés (fillFormForEdit)

A táblázat egy sorának "Szerkeszt" gombjára kattintva a rendszer előre kitölti az űrlapot a kiválasztott termék jelenlegi adataival.

Mentés — létrehozás és módosítás egy függvényben (handleSaveProduct)

```
const url = id ? `${API_BASE}/Product/${id}` : `${API_BASE}/Product`;
const method = id ? "PUT" : "POST";
```

Ha az űrlapon szerepel termék-id, a rendszer PUT kéréssel frissíti a meglévő terméket; ha nincs kitöltve, POST kéréssel új terméket hoz létre.

Duplikátum-ellenőrzés új termék hozzáadásakor (handleAddAsNew)

Mielőtt a rendszer új terméket hozna létre, összeveti a beírt adatokat a már meglévő termékekkel; ha talál egy mindenben megegyező terméket, megerősítést kér az adminisztrátortól, mielőtt a mentés megtörténne. Ez a véletlen duplikátumok létrehozása ellen véd.

3.2.9. Egyéb funkciók

Közvetlen hivatkozás egy adott termékre (URL-paraméter)

A Termékek.html oldal a betöltéskor ellenőrzi, hogy a URL tartalmaz-e egy termék nevű lekérdezési paramétert (pl. Termékek.html?termek=5). Ha igen, automatikusan megnyitja az adott termék részletes nézetét. Ezt a mechanizmust használja ki a nepszeruTermékek.js is: egy kiemelt termékre kattintva a felhasználó a Termékek oldalra kerül, és a kattintott termék részletes nézete azonnal megnyílik.

Kiemelt ("népszerű") termékek — nepszeruTermékek.js

A kezdőlapon megjelenő kiemelt termékek listája jelenleg kézzel, egy rögzített azonosító-listával (NEPSZERU_ID_LISTA) van megadva a kódban, nem automatikus (pl. eladási statisztikán alapuló) kiválasztás alapján. A megjelenítéshez a szokásos /Product végpontot használja, és a kapott listából szűri ki a megadott azonosítójú termékeket.

3.2.10. Ismert korlátok és továbbfejlesztési lehetőségek

- Az API alap-URL-je (API_BASE / AUTH_API_BASE) több fájlban külön-külön van megadva ugyanazzal az értékkel; egy közös, egyszer definiált konfigurációs fájlba érdemes lenne kiszervezni, hogy a backend címének változtatásakor ne kelljen több helyen módosítani.
- A kiemelt ("népszerű") termékek listája jelenleg kézzel karbantartott azonosító-lista; a jövőben automatizálható lenne például eladási adatok vagy admin általi kiemelés alapján.
- A frontend nem végez saját, kliensoldali bemeneti validációt a backend válaszára túl (pl. admin termék-úrlap kitöltöttsége); ez a jelenlegi állapotban a backend hibaüzeneteire támaszkodik.
- A felhasználói felület jövőbeli fejlesztési lehetőségei:
 - Szállítási és számlázási adatok megadására szolgáló űrlap a fiók/kosár oldalon
 - Bankkártyás fizetési felület integrálása a rendelés leadásának folyamatába
 - „Korábbi rendeléseim” nézet a felhasználói fiókban, rendelésállapot megjelenítésével

3.2.11. Hibakezelés a frontenden

A frontend kódja konzisztensen try/catch blokkokkal veszi körül a backend felé irányuló hálózati kéréseket. Hiba esetén kétféle dolog történik egyszerre:

- A hiba technikai részletei a böngésző fejlesztői konzoljára kerülnek kiírásra (`console.error(error)`), ami a fejlesztők számára segít a hibakeresésben.
- A felhasználó egy érthető, magyar nyelvű üzenetet lát a felületen (pl. "Hibás email vagy jelszó.", "Nem sikerült kapcsolódni a szerverhez.", "A kosarad üres."), a technikai részletek elrejtése mellett.

A backend válaszána stáuszará alapján eltérő felhasználói élmény jelenik meg, például:

- Sikertelen bejelentkezés (nem 200-as válasz) → hibaüzenet jelenik meg az űrlapon, az oldal működőképes marad.
- Rendelés leadásakor kapott 401-es válasz (nincs bejelentkezve) → a Kosar.html oldal nem hibaüzenetet ad, hanem helyben felkínálja a bejelentkezés lehetőségét, a kosár tartalmának megtartásával
- Termékek betöltésének sikertelensége (`indexBackend.js`, `nepszeruTermek.js`) → a termékek helyén egy tájékoztató szöveg jelenik meg ("Nem sikerült betölteni a termékeket. Próbáld újra később.") az üres vagy hibás megjelenítési felület helyett

Törlés megerősítése: a felhasználó véletlen adatvesztés elleni védelemként, mielőtt egy termék (`admin.js`) vagy kosártétel (`kosar.js`) törlődne, a böngésző natív megerősítő ablaka (`confirm()`) jelenik meg.

3.3. Adatbázis

Az alkalmazás egy **MySQL** adatbázist használ (projectwebshop, localhost:3306), amit a fejlesztés során **WAMP** szerverrel futtattunk, és **phpMyAdmin** felületen keresztül kezeltünk. Az adatbázis 4 táblát tartalmaz: Users, Products, Orders, OrderItems, amelyek InnoDB motorral, valódi idegen kulcs (FOREIGN KEY) megkötésekkel épülnek egymásra.

A backend a Connect.cs osztályon és a MySqlConnectionor NuGet csomagon keresztül kapcsolódik az adatbázishoz; minden API-hívás egy önálló, nyitott-majd-bezárt kapcsolatot használ (részletesen lásd a Backend fejlesztés **3.1.5. Kapcsolódás (adatbázis)** fejezetét).

3.3.1. Táblák szerkezete

A táblák pontos oszlopszerkezetét lásd a **2.5 Adatmodell** fejezetben.

Összefoglalva:

Tábla	Szerepe
Users	Regisztrált felhasználók adatai, hash-elt jelszóval és admin-jelöléssel
Products	A webshopban elérhető termékek
Orders	A leadott rendelések feadatai (ki, mikor, mennyiért)
OrderItems	Az egyes rendelések tételei (melyik termékből mennyit)

3.3.2. Kapcsolatok (FOREIGN KEY megkötések)

A fejlesztés során felmerült egy fontos technikai probléma és annak megoldása, amit érdemes dokumentálni:

A probléma: a táblák eredetileg **MyISAM** tárolómotorral jöttek létre. Amikor megpróbáltunk FOREIGN KEY megkötéseket hozzáadni, a MySQL hibaüzenet nélkül "elfogadta" a parancsot, de a phpMyAdmin Tervező nézete nem mutatott kapcsolatokat — kiderült, hogy a **MyISAM motor egyáltalán nem támogatja és nem kényszeríti ki** a FOREIGN KEY megkötéseket, csak szintaktikailag elfogadja őket.

A megoldás: a táblákat át kellett konvertálni **InnoDB** motorra (ami támogatja a hivatkozási integritást), majd újra hozzáadni a megkötéseket:

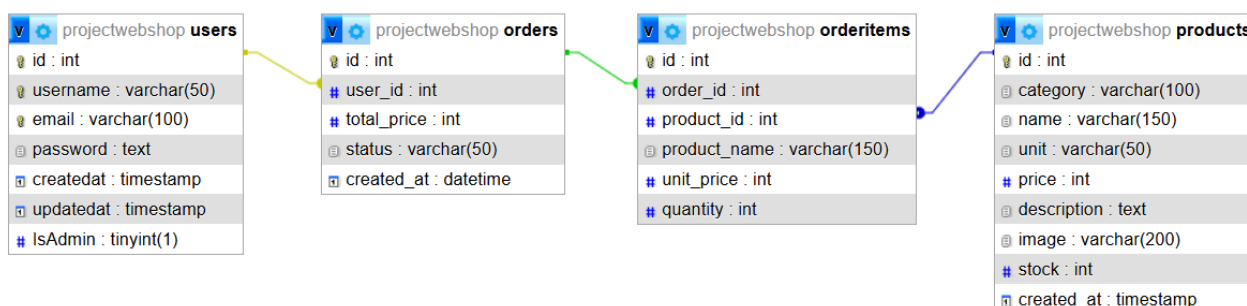
```
ALTER TABLE Users ENGINE=InnoDB;
ALTER TABLE Products ENGINE=InnoDB;
ALTER TABLE Orders ENGINE=InnoDB;
ALTER TABLE OrderItems ENGINE=InnoDB;
ALTER TABLE OrderItems MODIFY product_id INT NULL;
ALTER TABLE Orders
ADD FOREIGN KEY (user_id) REFERENCES Users(id) ON DELETE CASCADE;
ALTER TABLE OrderItems
ADD FOREIGN KEY (order_id) REFERENCES Orders(id) ON DELETE CASCADE;
ALTER TABLE OrderItems
ADD FOREIGN KEY (product_id) REFERENCES Products(id) ON DELETE SET NULL;
```

A megkötések és indoklásuk:

Kapcsolat	Viselkedés törléskor	Indoklás
Orders.user_id → Users.id	ON DELETE CASCADE	Ha egy felhasználó törlődik, a rendelései is törlődjenek vele
OrderItems.order_id → Orders.id	ON DELETE CASCADE	Ha egy rendelés törlődik, a tételei is törlődjenek vele
OrderItems.product_id → Products.id	ON DELETE SET NULL	Ha egy terméket törölnek (admin funkció), a korábbi rendelések tétele megmaradjon , csak a hivatkozás váljon üressé — mivel a product_name/unit_price mezők másolatként külön is tárolva vannak, a rendelési előzmény így is teljes marad

Egy adattisztítási lépés is szükséges volt: a megkötések hozzáadása előtt egy hibás adatra derült fény — egy Orders sor egy már nem létező felhasználóra (user_id) hivatkozott. Ezt egy ellenőrző lekérdezéssel (SELECT * FROM Orders WHERE user_id NOT IN (SELECT id FROM Users)) azonosítottuk, majd töröltük az érintett sort, mielőtt a megkötés sikeresen létrejöhetett.

3.3.4. ER-diagram



3.3.5. Adatműveletek (CRUD) bemutatása

A termékek kezelése (ProductController) jól szemlélteti, hogyan valósulnak meg az alap adatbázis-műveletek:

Olvasás (Read):

```
var cmd = new MySqlCommand("SELECT * FROM Products", connection);
```

Az összes termék lekérdezése, majd C# objektumokká alakítása egy MySqlDataReader segítségével.

Létrehozás (Create):

```
var cmd = new MySqlCommand(
    "INSERT INTO Products (category, name, unit, price, description, image, stock) " +
    "VALUES (@category, @name, @unit, @price, @description, @image, @stock)", connection);
```

Módosítás (Update) és Törlés (Delete): hasonlóan, paraméterezett UPDATE/DELETE SQL parancsokkal, admin jogosultsághoz kötve ([Authorize(Roles = "Admin")]).

3.3.6. Tranzakciókezelés

A rendelés leadása (OrderController) több táblát érintő, összetett művelet, ezért **tranzakción belül** fut, kódrészlet:

```
using var transaction = await connection.BeginTransactionAsync();
try
{
    // Orders sor létrehozása
    // OrderItems sorok létrehozása
    await transaction.CommitAsync();
}
catch (Exception ex)
{
    await transaction.RollbackAsync();
    return StatusCode(500, ...);
}
```

Ez biztosítja az **atomicitást**: egy rendelés vagy **teljes egészében** sikeresen mentésre kerül (a fejkörök és minden tétel), vagy hiba esetén **semmi** nem marad az adatbázisban félkész állapotban.

3.3.7. Biztonsági és adatintegritási szempontok

- A jelszavak BCrypt algoritmussal hash-elve tárolódnak, soha nem sima szövegként
- A Users.email mező egyedi kell legyen (regisztrációkor ellenőrizve alkalmazás-szinten)
- A FOREIGN KEY megkötések hozzáadásával az adatbázis motor szintjén is biztosított a hivatkozási integritás, nem csak az alkalmazás logikájában

3.3.8. További fejlesztési lehetőségek:

A jövőbeli funkcióbővítés (szállítás/számlázási adatok, online fizetés) új táblákat és oszlopokat igényelne, például:

- ShippingAddresses tábla (user_id, irányítószám, város, utca, házszám)
- BillingAddresses tábla (user_id, irányítószám, számlázási név, adószám-opcionális, cím)
- Payments tábla (order_id, fizetési mód, tranzakció azonosító, státusz)
- Orders tábla bővítése egy status oszloppal (feldolgozás alatt/kiszállítva/teljesítve)

3.4 Verziókezelés

A projekt forráskódját **Git**-tel verziókövetjük, a **GitHub**-on tárolt közös csapat-repository-ban (**Henriett1990/projektmunka2026**), amihez minden csapattag hozzáféréssel rendelkezik.

Munkafolyamat:

A fejlesztés során az alábbi lépéssorozatot követtük minden munkaszakasz végén:

git add.

git commit -m "Rövid leírás a változtatásról"

git pull origin main

git push origin main

Branch-stratégia:

A projekt mérete és a csapat létszáma miatt egyetlen közös main ágon dolgoztunk, közvetlen commitokkal — külön feature-branch-eket nem használtunk, mivel a párhuzamos munka viszonylag ritkán ütközött, és a rendszeres pull/push ciklus elegendő volt az összehangoláshoz.

Mit tartalmaz a verziókezelő tartalom:

- A teljes frontend forráskód (HTML, CSS, JavaScript fájlok)
- A teljes backend forráskód (C# projekt, Controllers, Models, Dtos)
- Az adatbázis szerkezetének és tartalmának exportált SQL fájljai (schema.sql, data.sql), amelyek lehetővé teszik, hogy bárki a csapatból újra létre tudja hozni ugyanazt az adatbázist a saját gépén

4. Tesztelés

A fejlesztés során **manuális tesztelés** történt, automatizált (unit/integrációs) tesztek nem készültek. A tesztelés a fejlesztéssel párhuzamosan, folyamatosan zajlott: minden újonnan elkészült funkciót (regisztráció, bejelentkezés, kosár, rendelés leadása, admin termékkezelés) közvetlenül a böngészőben, valós felhasználói forgatókönyvek végigjátszásával ellenőriztünk. A tesztelés kiterjedt a normál (helyes bemenetekkel történő) működésre, valamint a hibás/szélsőséges esetekre is (pl. hibás jelszó, üres kosár, bejelentkezés nélküli hozzáférési kísérlet védett funkciókhoz), hogy a rendszer megfelelően kezelje ezeket is. A böngésző fejlesztői eszközeit (Console, Network fül) is rendszeresen használtuk a hibák felderítéséhez és a háttérben zajló API-kommunikáció ellenőrzéséhez.

4.1. Tesztesetek

Azonosító	Leírás	Elvárt eredmény	Tényleges eredmény	Sikeres?
T001	Regisztráció működése	Sikeres regisztráció, 201 státusz	A regisztráció sikeresen lezajlott, 201-es státuszkóddal, az új felhasználó megjelent az adatbázisban	Igen
T002	Regisztráció már foglalt email címmel	Hibaüzenet, 409 Conflict státusz	409-es hibaüzenet jelent meg ("Ez az email cím már regisztrálva van."), a regisztráció nem ment végbe	Igen
T003	Bejelentkezés helyes email/jelszó párossal	Sikeres bejelentkezés, session-cookie kiállítva	Sikeres bejelentkezés, session-cookie kiállítva, nav "Fiókom"/"Kijelentkezés"-re váltott	Igen
T004	Bejelentkezés hibás jelszóval	401 Unauthorized, hibaüzenet megjelenik	401-es státuszkódot adott vissza a szerver, a felületen megjelent a "Hibás email vagy jelszó." hibaüzenet	Igen
T005	Bejelentkezés nem létező email címmel	401 Unauthorized, hibaüzenet megjelenik	401-es státuszkódot adott vissza a szerver, a felületen megjelent a "Hibás email vagy jelszó." hibaüzenet	Igen

T006	Kijelentkezés	A session megszűnik, a nav "Bejelentkezés"-re vált vissza	Sikeres kijelentkezés után a nav sávban ismét a "Bejelentkezés" felirat jelent meg, a session megszűnt	Igen
T007	Bejelentkezés Enter billentyűvel (gombkattintás nélkül)	A bejelentkezés lefut, ugyanúgy, mint gombnyomásra	Enter billentyűvel is lefutott a bejelentkezés	Igen
T008	Termékek listázása bejelentkezés nélkül	A termékek megjelennek, hozzáférés nem korlátozott	Bejelentkezés nélkül is megjelent a teljes termékkínálat a Termékek oldalon	Igen
T009	Egy termékre kattintás	Megnyílik a részletes nézet, teljes leírással	A termékre kattintva megnyílt a részletes nézet doboz, a teljes leírással, árral és kiszerezéssel	Igen
T010	Termék hozzáadása a kosárhoz	A termék bekerül a kosárba, megjelenik a visszajelzés	A termék bekerült a kosárba, toaster üzenet megjelent	Igen
T011	Kosárban lévő tétel mennyiségének módosítása	A részösszeg és a végösszeg automatikusan frissül	Mennyiség módosításkor a végösszeg frissült	Igen
T012	Tétel törlése a kosárból	A tétel eltűnik a listából, a végösszeg frissül	Tétel törlésekor a lista és végösszeg frissült	Igen
T013	Kosár tartalmának megőrzése oldal-újrátöltés után	A kosár tartalma megmarad (localStorage)	Oldal-újrátöltés után a kosár tartalma (termékek, mennyiségek) változatlanul megmaradt	Igen
T014	Rendelés leadása bejelentkezés nélkül	Hibaüzenet jelenik meg, "Bejelentkezés" gomb felajánlva	Bejelentkezés nélkül hibaüzenet + "Bejelentkezés" gomb jelent meg	Igen
T015	Rendelés leadása bejelentkezés nélkül, nem üres kosárral	Sikeres rendelés, visszaigazoló üzenet, kosár kiürül	Sikeres rendelés, "Rendelésed sikeresen leadva" üzenet, kosár kiürült	Igen
T016	Email érkezése rendelés leadása után (vásárlói cím)	A vásárló email címére megérkezik a visszaigazolás	Vásárlói email megérkezett	Igen

T017	Email érkezése rendelés leadása után (admin cím)	Az admin email címére megérkezik az értesítés	Admin email megérkezett	Igen
T018	Rendelés leadása üres kosárral	Hibaüzenet: "A kosarad üres."	Megjelent a "A kosarad üres." hibaüzenet, a rendelés nem került leadásra	Igen
T019	Admin bejelentkezés után a termékkezelő felület elérése	Megjelenik a "Termékek kezelése" nézet	Admin bejelentkezés után megjelent a termékkezelő felület	Igen
T020	Új termék hozzáadása admin felületen	Az új termék megjelenik a listában és a vásárlói oldalon is	Új termék hozzáadva, megjelent a listában	Igen
T021	Meglévő termék szerkesztése	A módosított adatok elmentődnek és megjelennek	A módosított termékadatok elmentődtek, és frissülve jelentek meg mind az admin táblázatban, mind a vásárlói oldalon	Igen
T022	Termék törlése	A termék eltűnik a listából és a vásárlói oldalról	A törölt termék eltűnt az admin táblázatból, és a vásárlói Termékek oldalon sem jelent meg többé	Igen
T023	Nem admin felhasználó megpróbál terméket módosítani (API szinten)	403 Forbidden / elutasítás	A kérés elutasításra került, admin jogosultság nélkül nem sikerült terméket létrehozni/módosítani	Igen
T024	Termékkezelő végpontok elérése bejelentkezés nélkül	401 Unauthorized	Bejelentkezés nélkül a termékkezelő végpontok 401-es hibát adtak vissza, a művelet nem sikerült	Igen
T025	Weboldal megjelenése mobil képernyőméretben	A menü hamburger-nézetbe csukódik, a tartalom reszponzív marad	Kisebb nézetben a menü hamburgerre váltott, reszponzív maradt	Igen
T026	Termék szerkesztése - negatív ár megadása	A rendszer elutasítja a mentést és hibaüzenet jelenik meg	A mentés sikertelen és az alábbi hibaüzenet jelenik meg: „Az ár nem lehet negatív”	Igen

4.3. Hibák listája

Fejlesztés közben ténylegesen előfordult hibák:

Azonosító	Leírás	Állapot	Megjegyzés
H001	CORS hiba: a frontend nem tudott kéréseket küldeni a backendnek	Javítva	A frontend és backend külön origin-ről fut (5501 vs 7249 port) — AddCors/UseCors policy hozzáadva a Program.cs-ben
H002	Bejelentkezés után a session-cookie nem maradt meg, minden kérés 401-et adott	Javítva	A cookie SameSite=Lax alapbeállítása blokkolta a cross-origin kéréseket — SameSite=None és SecurePolicy=Always oldotta meg
H003	A "Kosárba" gomb hover-színe nem a tervezett árnyalatra váltott	Javítva	Egy duplikált .btn-kosarba: hover CSS szabály felülírta a kívánt beállítást — a felesleges, régi szabály törölve
H004	Egy termékkép nem töltődött be, miközben a többi igen	Javítva	Elgépelés az adatbázisban tárolt kép-elérési útban (hiányzó "d" betű a fájlnevében) — a helyes érték beállítva phpMyAdminben
H005	A UserController regisztrációs végpontja soha nem mentett adatot	Javítva	Felfedezve, hogy ez egy korai fejlesztési szakaszból megmaradt, funkció nélküli csonk kód volt — eltávolítva, mivel a tényleges regisztráció az AuthController.Register()-ben valósul meg
H006	A navigációs sávból eltűnt a "Kezdőlap" menüpont, a "Bejelentkezés" pedig duplán jelent meg	Javítva	Egy azonosító (id="auth-nav-item") véletlenül a rossz elemre került — a nav szerkezet javítva
H007	JavaScript hiba: "Unexpected token" a rendelés-leadási logikában	Javítva	Egy felesleges extra záró kapcsos zárójel (}) törte meg az if/else if/else láncot a rendelesLeadasa() függvényben — eltávolítva
H008	Egyes termékképek nem jelentek meg	Javítva	A fájlnevekben lévő speciális karakterek (szóköz, %) nem voltak URL-kódolva — encodeURI() alkalmazása megoldotta
H009	FOREIGN KEY megkötések hozzáadása "sikeresnek" tűnt, de nem jelentek meg kapcsolatok	Javítva	A táblák MyISAM motort használtak, ami nem támogatja a FOREIGN KEY-eket — InnoDB-re konvertálva
H010	FOREIGN KEY hozzáadása hibát dobott (#1452)	Javítva	Egy Orders rekord érvénytelen user_id-ra hivatkozott — az árva rekord törölve, ezután sikeres volt a megkötés létrehozása
H011	A nav sávban a bejelentkezési állapot nem frissült oldal-újrátöltés nélkül	Javítva	A nav-frissítő függvény hívása hiányzott a sikeres bejelentkezés utáni logikából — pótolva

5. Felhasználói kézikönyv

5.1 Bevezetés

Ez a kézikönyv bemutatja a szoftver működését a végfelhasználók számára. A webshop lehetővé teszi a kecsketejalapú és kenderolajos termékek böngészését, kosárba helyezését és megrendelését, valamint bejelentkezett adminisztrátorok számára a termékkínálat kezelését. Az oldal tetején elhelyezkedő navigációs sáv (Kezdőlap, Termékek, Rólunk, Elérhetőségek, Általános feltételek, Bejelentkezés/Fiókom, Kosár) minden aloldalon elérhető.

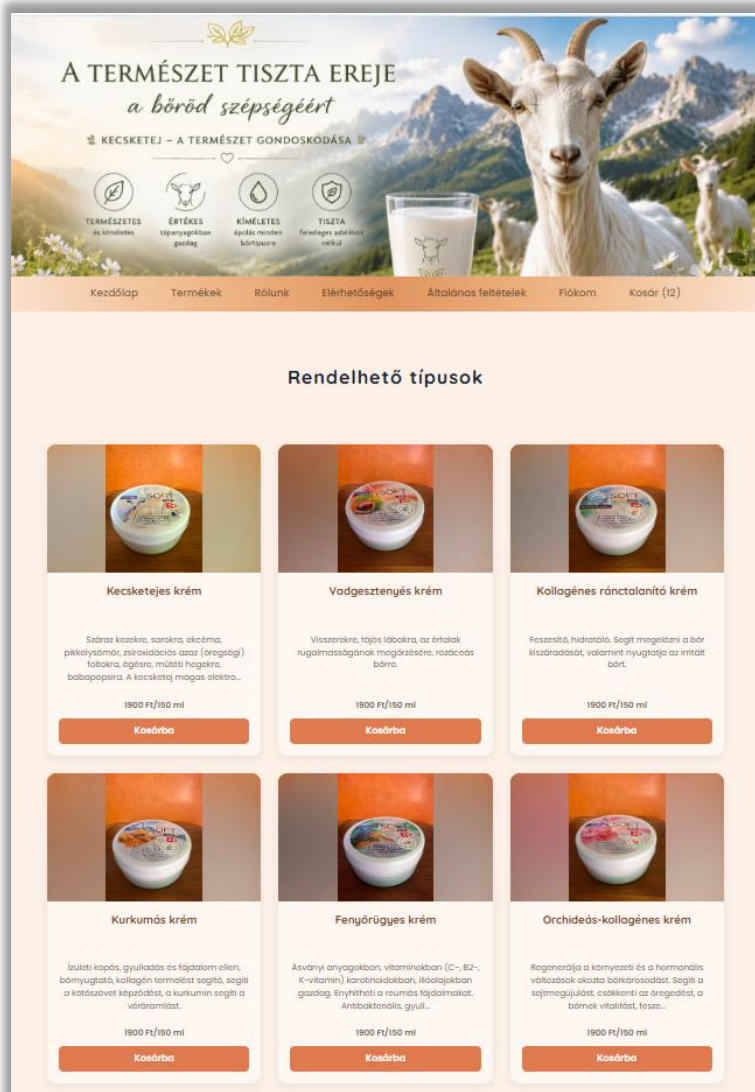
5.2 Kezdőlap

A weboldal megnyitásakor a **Kezdőlap** jelenik meg elsőként. Itt található:

- Egy váltakozó képes fejléc ("A természet ereje, minden cseppben" felirattal), ami a márka jellegét (természetesség, minőség) mutatja be
- Egy Bemutakozás szakasz, amely röviden ismerteti a vállalkozás történetét (2008 óta forgalmazott kecsketejalapú és kenderolajos termékek)
- Egy Népszerű termékek szekció, ahol a legkeresettebb termékek kártyás formában jelennek meg, mindegyik saját "Kosárba" gombbal, és a termék képére kattintva a részletes nézet is megnyitható



5.3 Termékek böngészése



A **Termékek** menüpontra kattintva megjelenik a teljes termékkínálat, kártyás elrendezésben. Minden kártya tartalmazza:

- A termék képét
- A termék nevét
- A rövidített leírását
- Az árát és kiszárazását
- Egy "Kosárba" gombot

Ha a vásárló egy termék képre vagy nevére kattint, egy részletes nézet nyílik meg a lista felett, ami a teljes (nem rövidített) leírást is megmutatja.

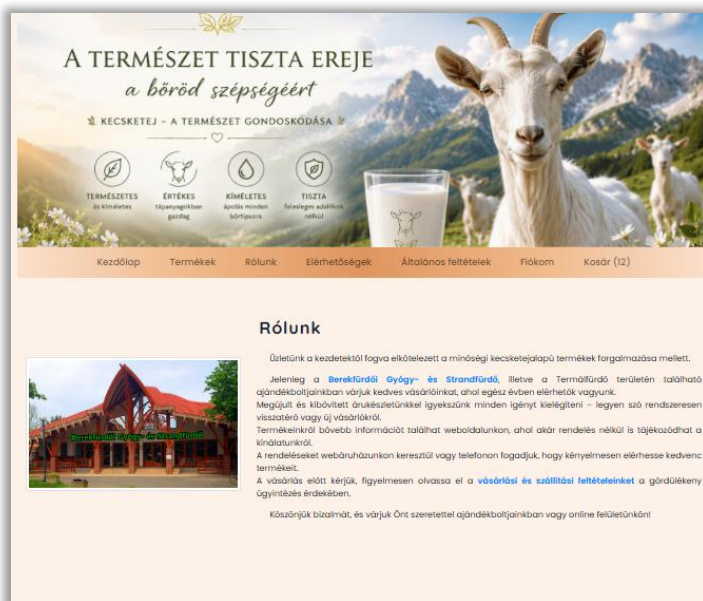


5.4. Rólunk

A **Rólunk** oldal a vállalkozás bemutatkozását és jelenlegi üzleteinek elérhetőségét ismerteti.

Itt található:

- Egy kép a Berekfürdői Gyógy- és Strandfürdőről, ahol az üzlet található
- Egy szöveges leírás a vállalkozás jelenlegi tevékenységéről és ajánlékboltjainak elérhetőségéről
- Hivatkozás a Vásárlási és szállítási feltételek oldalra



5.5. Elérhetőségek

Az **Elérhetőségek** oldal a vállalkozás kapcsolattartási adatait mutatja be.

Itt található:

- Egy interaktív térkép az üzlet helyének megjelölésével
- Cím és telefonos elérhetőség
- Hivatkozás az üzlet email címével



5.6. Általános feltételek



Ez az oldal a webshopon keresztül történő vásárlás szabályait foglalja össze.

Itt található:

- A rendelés menete
- Szállítási módok és díjak
- Garanciára és elállásra vonatkozó tudnivalók
- Vevőszolgálati elérhetőség panaszkezeléshez

5.7. Regisztráció és bejelentkezés

A navigációs sáv "**Bejelentkezés**" menüpontjára kattintva egy egyszerű form jelenik meg, email cím és jelszó mezőkkel. Ha a felhasználónak még nincs fiókja, a "Nincs még fiókod? Regisztrálj!" linkre kattintva a form regisztrációs módba vált, ahol felhasználónevet, email címet és jelszót kell megadni.

Sikeres bejelentkezés után a menüpont felirata "Fiókom"-ra változik, ahol egy "Kijelentkezés" gomb is elérhető. (Ez a nézet később bővül majd a saját kosár/rendelési előzmények megjelenítésével.)

A Bejelentkezés oldalon a regisztrált felhasználók léphetnek be fiókjukba email címmel és jelszóval. Ha a felhasználónak még nincs fiókja, ugyanerről a felületről regisztrálhat is — a rendszer egyetlen úrlapon, egy gombbal vált a bejelentkezési és regisztrációs nézet között.



5.8. Kosár használata

A jobb felső sarokban lévő **"Kosár (N)"** felirat mindig mutatja, hány tétel van jelenleg a kosárban.

A Kosaram oldalon:

- Táblázatos formában látszik minden kosárba tett termék, egységgel, mennyiséggel és részösszeggel
- A mennyiség mező közvetlenül szerkeszthető, a végösszeg automatikusan újraszámolódik
- A "Törlés" gombbal egy-egy tétel eltávolítható a kosárból
- Alul látható a Végösszeg, és a "Rendelés leadása" gomb

Termék	Egységár	Mennyiség	Részösszeg	
Kecskefejes krém	1900 Ft/150 ml	2	3800 Ft	Törlés
Vadgesztenyes krém	1900 Ft/150 ml	2	3800 Ft	Törlés
Kollagénes ráncatlanító krém	1900 Ft/150 ml	2	3800 Ft	Törlés
Fenyőrózsa krém	1900 Ft/150 ml	2	3800 Ft	Törlés
Kurkumás krém	1900 Ft/150 ml	2	3800 Ft	Törlés
Orchideás-kollagénes krém	1900 Ft/150 ml	2	3800 Ft	Törlés

Végösszeg: 22800 Ft

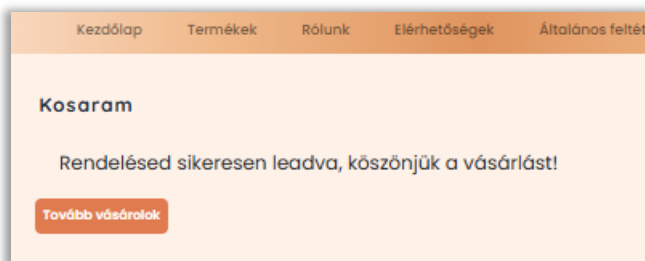
[Rendelés leadása](#)

A kosár tartalma a böngészőben (localStorage) tárolódik, tehát oldal-újratöltés után is megmarad, bejelentkezés nélkül is.

5.9. Rendelés leadása

A Kosár oldalon a "**Rendelés leadása**" gombra kattintva:

- Ha a vásárló be van jelentkezve, a rendelés rögzítésre kerül, a kosár kiürül, és egy visszaigazoló üzenet jelenik meg ("Rendelésed sikeresen leadva, köszönjük a vásárlást!"), a folytatáshoz pedig egy "Tovább vásárolok" gomb vezet vissza a Termékek oldalra



- A sikeres rendelésről a vásárló és az admin automatikusan visszaigazoló emailt kap



- Ha a vásárló nincs bejelentkezve, egy figyelmeztető üzenet és egy beépített bejelentkező form jelenik meg közvetlenül a kosár oldalon, hogy ne kelljen elnavigálnia máshova

Fontos, hogy egy termékből csak annyi darab rendelhető, amennyi az adott pillanatban készleten van – ha a kosárban megadott mennyiség meghaladja az elérhető készletet, a rendszer hibaüzenetben jelzi ezt, és a rendelés nem kerül feldolgozásra.

Kosaram

Termék	Egységár	Mennyiség	Részösszeg	
Kecsketejes krém	1900 Ft/150 ml	1	1900 Ft	Törlés
Vadgesztenyes krém	1900 Ft/150 ml	35	66500 Ft	Törlés

Végösszeg: 68400 Ft

'Vadgesztenyes krém' termékből csak 16 db van készleten, de 35 db-ot próbáltál rendelni.

Rendelés leadása

5.10. Admin funkciók

Bejelentkezett adminisztrátor számára a "Fiókom" helyett a "Termékek kezelése" felület jelenik meg.

Itt található:

- A bejelentkezett felhasználó neve és egy "Kijelentkezés" gomb
- Egy "Termék hozzáadása / módosítása" form (Kategória, Név, Mennyiség/egység, Ár, Leírás, Kép elérési útja, Készlet mezőkkel), alatta három gombbal:
 - ☐ "Mentés" — a jelenleg szerkesztett (kiválasztott) termék módosításainak elmentése
 - ☐ "Új termék" — a form tartalmának mindig új termékként való hozzáadása
 - ☐ "Mégse" — a form kiürítése

Termékek kezelése

Bejelentkezve: tesztelek

Kijelentkezés

Termék hozzáadása / módosítása

Kategória

Név

Mennyiség/egység (pl. 150 ml)

Ár (Ft)

Leírás

Kép elérési útja

Készlet

Mentés
Új termék
Mégse

Termékek listája

ID	Kategória	Név	Egység	Ár	Készlet	Műveletek
1	Soft	Kecsketejes krém	150 ml	1900	0	Szerkeszt Törlés
2	Soft	Vadgesztenyes krém	150 ml	1900	0	Szerkeszt Törlés

- "Termékek listája" táblázat, ami az összes terméket felsorolja (ID, Kategória, Név, Egység, Ár, Készlet oszlopokkal), mindegyik mellett "Szerkeszt" és "Törlés" gombbal — a "Szerkeszt" gombra kattintva a termék adatai betöltődnek a fenti formba és szerkeszthetővé válik.

Egy kitöltött form:

Termékek kezelése

Bejelentkezve: tesztelek

Kijelentkezés

Termék hozzáadása / módosítása

Kategória

Név

Mennyiség/egység (pl. 150 ml)

Ár (Ft)

Leírás

Száraz kezekre, sarokra, ekcéma, pikkelysömör, zsíroxidációs azaz (öregségi) foltokra, égésre, műtési hegekre, babapopsira. A kecsketej magas elektrolit tartalmának köszönhetően kiválóan alkalmas a gyulladt, irritált bőr

Kép elérési útja

Készlet

Mentés **Új termék** **Mégse**

Termékek listája

ID	Kategória	Név	Egység	Ár	Készlet	Műveletek
1	Soft	Kecskekejes krém	150 ml	1900	0	Szerkeszt Törölés

A termék ára nem lehet negatív. Ha az admin mégis negatív árat ad meg, akkor egy üzenet figyelmezteti erre a felhasználót és a változtatások nem lesznek elmentve:

Ár (Ft)

Leírás

Száraz kezekre, sarokra, ekcéma, pikkelysömör, zsíroxidációs azaz (öregségi) foltokra, égésre, műtési hegekre, babapopsira. A kecsketej magas elektrolit tartalmának köszönhetően kiválóan alkalmas a gyulladt, irritált bőr

Kép elérési útja

Készlet

Mentés **Új termék** **Mégse**

Az ár nem lehet negatív.